

Управління програмними проектами

Моделі (методології) розробки
програмного забезпечення

Моделі (або методології) процесів розробки ПЗ прийнято класифікувати за «вагою» — кількості формалізованих процесів (більшість процесів або тільки основні) і детальності їх регламентації. Чим більше процесів документовано, ніж більш детально вони описані, тим більше «вага» моделі.

СТАНДАРТИ

Одним з перших стандартів, що мав істотний вплив на розвиток теорії проектування та розробки ІС, був стандарт BSP (Business System Planning). Даний стандарт був розроблений компанією IBM в середині 70-х рр. Процес BSP передбачав виділення в ході розробки ІС таких кроків: — отримання підтримки керівництва; — визначення процесів підприємства; — визначення класів даних; — проведення інтерв'ю; — обробка та організація результатів інтерв'ю. Найважливіші кроки процесу BSP спостерігаються у більшості формальних методик.

На сьогодні діють такі стандарти, що регламентують процес розробки ПЗ:

ГОСТ 34.601-90 - державний стандарт, поширюється на автоматизовані системи і встановлює стадії і етапи їх створення. У стандарті міститься опис змісту робіт на кожному етапі.

ISO / IEC 12207 - Systems and software engineering - Software Life Cycle Processes - міжнародний стандарт на процеси розробки та організації життєвого циклу ПЗ. Поширюється на всі види замовленого ПЗ.

SWEBOOK (Guide to the Software Engineering Body of Knowledge) - Посібник до зведення знань з програмної інженерії - галузевий стандарт Інституту інженерів по радіоелектроніці та електротехніці (IEEE), що містить основні види діяльності з програмної інженерії.

- ДСТУ 3008-95 „Документація. Звіти у сфері науки і техніки. Структура і правила оформлення”;
- ДСТУ 3973-2000 „Правила виконання науково-дослідних робіт. Загальні положення”;
- ДСТУ 3974-2000 „Правила виконання дослідно-конструкторських робіт. Загальні положення”;
- ДСТУ 3396.0-96 Захист інформації. Технічний захист інформації. Основні положення;
- ДСТУ 3396.1-96 Захист інформації. Технічний захист інформації. Порядок проведення робіт;
- ДСТУ 3396.2-97 Захист інформації. Технічний захист інформації. Терміни та визначення;
- ДСТУ 2844-94 Програмні засоби ЕОМ. Забезпечення якості. Терміни та визначення;
- ДСТУ 2873-94 Системи оброблення інформації. Програмування. Терміни та визначення;
- ДСТУ 2941-94 Системи оброблення інформації. Розроблення систем. Терміни та визначення;
- ДСТУ ISO/IEC 2382-15:2005 Інформаційні технології. Словник термінів. Частина 15. Мови програмування;
- ДСТУ ISO/IEC 2382-5:2005 Інформаційні технології. Словник термінів. Частина 5. Подання даних;
- ДСТУ ISO/IEC 2382-4:2005 Інформаційні технології. Словник термінів. Частина 4. Організація даних;
- ДСТУ ISO/IEC 2382-17:2005 Інформаційні технології. Словник термінів. Частина 17. Бази даних;
- ДСТУ ISO/IEC 2382-18:2005 Інформаційні технології. Словник термінів. Частина 18. Розподілене оброблення даних;

- ДСТУ ISO/IEC 2382-9:2005 Інформаційні технології. Словник термінів. Частина 9. Обмін даними;
- ДСТУ ISO/IEC 2382-14:2005 Інформаційні технології. Словник термінів. Частина 14. Безвідмовність, ремонтпридатність і готовність;
- ДСТУ ISO/IEC 90003:2006 Програмна інженерія. Настанови щодо застосування ISO 9001:2000 до програмного забезпечення (ISO/IEC 9003:2004, IDT);
- ДСТУ 4071–2002 Інформаційні технології. Архітектура відкритого розподіленого керування та підтримка загальної архітектури брокера об'єктних запитів (CORBA);
- ДСТУ 4072–2002 Інформаційні технології. Мови програмування, їхні середовище і системний інтерфейс. Незалежний від мов виклик процедур (LIPC);
- ДСТУ ISO/IEC TR 14369:2003 Інформаційні технології. Мови програмування, їхні середовище і системний інтерфейс. Настанова щодо підготовки незалежних від мов специфікацій сервісу (LISS);
- ДСТУ 4249:03 Інформаційні технології. Настанова щодо POSIX-сумісних середовищ відкритих систем (POSIX-OSE) (ISO/IEC TR 14252:1996, MOD);
- ДСТУ 2850-94 Програмні засоби ЕОМ. Показники і методи оцінювання якості;
- ДСТУ 2851-94 Програмні засоби ЕОМ. Документування результатів випробувань;
- ДСТУ 2853-94 Програмні засоби ЕОМ. Підготовлення і проведення випробувань;
- ДСТУ 3918-99 (ISO/IEC 12207:1995) Інформаційні технології. Процеси життєвого циклу програмного забезпечення;
- ДСТУ 3919-99 (ISO/IEC 14102:1995) Інформаційні технології. Основні напрямки оцінювання та відбору CASE-інструментів;
- ДСТУ 4302:2004 Інформаційні технології. Настанови щодо документування комп'ютерних програм (ISO/IEC 6592:2000, MOD) ;
- ДСТУ ISO/IEC TR 12182:2004 Інформаційні технології. Класифікація програмних засобів (ISO/IEC TR 12182:1998, IDT) ;
- ДСТУ ISO/IEC 14598-1:2004 Інформаційні технології. Оцінювання програмного продукту. Частина 1. Загальний огляд (ISO/IEC 14598-1:1999, IDT) ;
- ДСТУ ISO/IEC 14598-2:2005 Інформаційні технології. Оцінювання програмного продукту. Частина 2. Планування та керування (ISO/IEC 14598-2:2000, IDT) ;
- ДСТУ ISO/IEC 14598-3:2005 Інформаційні технології. Оцінювання програмного продукту. Частина 3. Процес для розробників (ISO/IEC 14598-3:2000, IDT) ;
- ДСТУ ISO/IEC 14598-4:2005 Інформаційні технології. Оцінювання програмного продукту. Частина 4. Процес для замовників (ISO/IEC 14598-4:1999, IDT) ;
- ДСТУ ISO/IEC 14598-5:2005 Інформаційні технології. Оцінювання програмного продукту. Частина 5. Процес для оцінювачів (ISO/IEC 14598-5:1998, IDT) ;

- ДСТУ ISO/IEC 14598-6:2005 Інформаційні технології. Оцінювання програмного продукту. Частина 6. Документація модулів оцінювання (ISO/IEC 14598-6:2001, IDT);
- ДСТУ ISO/IEC 14764-2002 Інформаційні технології. Супровід програмного забезпечення (ISO/IEC 14764:1999, IDT);
- ДСТУ ISO/IEC 15288:2005 Інформаційні технології. Процеси життєвого циклу системи (ISO/IEC 15288:2002, IDT) ;
- ДСТУ ISO/IEC TR 15504-1-2002 Інформаційні технології. Оцінювання процесів життєвого циклу програмних засобів. Частина 1. Концепції та вступна настанова (ISO/IEC TR 15504-1:1998, IDT) ;
- ДСТУ ISO 9735-1:2006 Електронний обмін даними для адміністрування, у торгівлі і на транспорті (EDIFACT). Правила синтаксису прикладного рівня (номер версії синтаксису: 4, номер редакції синтаксису: 1); у 10 частинах.
- ДСТУ ISO/TS 20625:2007 Обмін електронними даними для управління, торгівлі і транспорту (EDIFACT). Правила генерації файлів XML-схем (XSD) на основі настанови з реалізації EDI(FACT);
- ДСТУ 4145:2002 Інформаційні технології. Криптографічний захист інформації. Електронний цифровий підпис, що ґрунтується на еліптичних кривих;
- ДСТУ ISO/IEC 13888–2002 Інформаційні технології. Методи захисту. Неспростовність»: Частина 1. Загальні положення;
- ДСТУ ISO/IEC 13888–2002 Інформаційні технології. Методи захисту. Неспростовність»: Частина 3. Механізми з використанням асиметричних методів
- ДСТУ ISO/IEC 14888–1:2002 Інформаційні технології. Методи захисту. Цифрові підписи з доповненням» Частина 1. Загальні положення
- ДСТУ ISO/IEC 14888–2:2002 Інформаційні технології. Методи захисту. Цифрові підписи з доповненням» Частина 2. Механізми на основі ідентифікаторів
- ДСТУ ISO/IEC 14888–3:2002 Інформаційні технології. Методи захисту. Цифрові підписи з доповненням» Частина 3. Механізми на основі сертифікатів
- ДСТУ ISO/IEC 10118-1:2000 Інформаційні технології. Методи захисту. Геш функції. Частина 1. Загальні положення
- ДСТУ ISO/IEC 10118-2:2000 Інформаційні технології. Методи захисту. Геш функції. Частина 2. Геш функції, що використовують n-бітний блоковий шифр

- ДСТУ ISO/IEC 10118-3:2004 Інформаційні технології. Методи захисту. Геш функції. Частина 3. Спеціалізовані геш функції
- ДСТУ ISO/IEC 13335-1:2004 Інформаційні технології. Методи захисту. Керування інформацією й безпекою технології комунікацій. Частина 1. Поняття й моделі для інформації й керування безпекою технології комунікацій
- ДСТУ ISO/IEC 15946-1:2008 Інформаційні технології. Методи захисту. Криптографічні методи, засновані на еліптичних кривих. Частина 1. Загальні положення
- ДСТУ ISO/IEC 18014-1:2002 Інформаційні технології. Методи захисту .Послуги штемпелювання часу - Частина 1: Структура
- ДСТУ ISO/IEC 18014-2:2002 Інформаційні технології. Методи захисту. Послуги штемпелювання часу. Частина 2. Механізми, що генерують незалежні токени
- ДСТУ ISO/IEC 9798-1:1997 Інформаційні технології. Методи захисту. Автентифікація сутності. Частина 1. Загальні положення
- ДСТУ ISO/IEC 9798-3:1998 Інформаційні технології. Методи захисту. Автентифікація сутності. Частина 3. Механізми , що використовують методи цифрового підпису
- ДСТУ ISO/IEC TR 13335-1:2001 Інформаційні технології. Настанова для керування ІТ безпекою. Частина 5. Настанова керування безпекою мережі
- ДСТУ-П CWA 14172-1:2008 Настанова EESSI з оцінювання відповідності. Частина 1: Загальні положення
- ДСТУ-П CWA 14172-2:2008 Настанова EESSI з оцінювання відповідності. Частина 2. Послуги та процеси органу сертифікації
- ДСТУ-П CWA 14172-3:2008 Настанова EESSI з оцінювання відповідності. Частина 3. Надійні системи, що управляють сертифікатами для електронних підписів
- ДСТУ-П CWA 14172-4:2008 Настанова EESSI з оцінювання відповідності. Частина 4. Застосовування для накладання підпису та загальні настанови з перевірки електронного підпису
- ДСТУ-П CWA 14172-5:2008 Настанова EESSI з оцінювання відповідності. Частина 5. Безпечні засоби створення підпису

Розробка у відповідності з цими стандартами проводиться за етапами, кожен з яких передбачає виконання певних робіт, і завершується випуском досить великого числа досить формалізованих і великих документів. Таким чином, суворе дотримання цих стандартів не тільки призводить до водоспадного підходу, але і вимагає дуже високого ступеня формалізованої розробки.

У 1963 в результаті злиття Інституту радіотехніки (Institute of Radio Engineers, IRE) і Американського інституту інженерів - електриків (American Institute of Electrical Engineers, AIEE) була створена міжнародна некомерційна асоціація технічних фахівців - світовий лідер в області розробки стандартів з радіоелектроніки та електротехніки – інститут інженерів з радіоелектроніки та електротехніки IEEE (Institute of Electrical and Electronics Engineers).

Дана міжнародна організація об'єднує понад 400 тисяч фахівців з 170 країн. IEEE здійснює інформаційну та матеріальну підтримку фахівців для організації та розвитку наукової діяльності в електротехніці, електроніці, комп'ютерній техніці та інформатиці.

IEEE здійснює інформаційну та матеріальну підтримку фахівців для організації та розвитку наукової діяльності в електротехніці, електроніці, комп'ютерній техніці та інформатиці.

SWEBOOK (Guide to the Software Engineering Body of Knowledge) містить опис 10 галузей знань:

Software requirements – програмні вимоги;

Software design – дизайн (архітектура);

Software construction – конструювання програмного забезпечення;

Software testing – тестування;

Software maintenance – експлуатація (підтримка) програмного забезпечення;

Software configuration management – управління конфігураціями;

Software engineering management – управління з програмної інженерії ;

Software engineering process – процеси програмної інженерії;

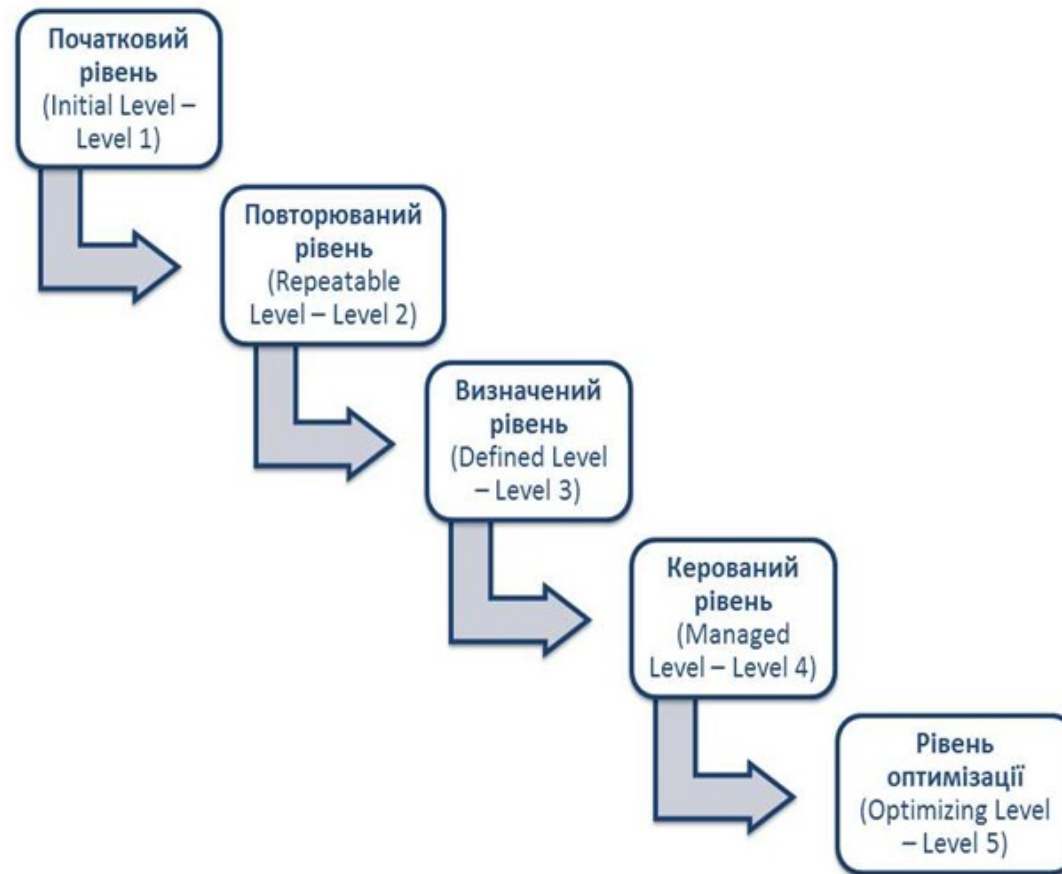
Software engineering tools and methods – інструменти та методи; Software quality— якість програмного забезпечення.

Для кожної галузі SWEBOOK містить опис ключових елементів у вигляді підобластей (subareas). Для кожної підобласті приведена декомпозиція в вигляді списку тем (topics) з їх описом.

У середині 80-х років минулого століття Міністерство оборони США міцно задумався про те, як вибирати розробників ПЗ при реалізації великих програмних проектів. За замовленням військових Інститут програмної інженерії, що входить до складу Університету Карнегі-Меллона, розробив **SW-CMM, Capability Maturity Model for Software**. У підсумку методологія виявилася надзвичайно корисною для більшості компаній, які прагнуть якісно поліпшити існуючі процеси проектування та розробки програмних засобів.. Надалі ця модель переросла в методологію підвищення якості процесів підприємства **Capability Maturity Model Integration CMMI**. Застосування CMMI дозволяє поставити розробку ПЗ на промислову основу, підвищити керованість ключовими процесами і виробничу культуру в цілому, гарантувати якісну роботу та виконання проектів в строк.

Основою для створення CMM стало базове положення про те, що фундаментальна проблема "кризи" процесу розробки якісного ПЗ полягає не у відсутності нових методів і засобів розробки, а в нездатності компанії організувати технологічні процеси і керувати ними.

Дана модель визначає п'ять рівнів зрілості процесу розробки ПЗ, за якими замовники можуть оцінювати потенційних претендентів на підпис контракту, а розробники - удосконалювати процеси створення ПЗ.



Початковий рівень (Initial Level - Level 1)

На даному рівні компанія може отримати замовлення, розробити і передати замовнику програмний продукт. Процес розробки носить хаотичний характер.. Стабільність розробок відсутня. Лише деякі процеси визначені, результат повністю залежить від зусиль окремих співробітників. Успіх одного проекту не гарантує успішності наступного. До цієї категорії можна віднести будь-яку компанію, яка хоч якось виконує взяті на себе зобов'язання.

Повторюваний рівень (Repeatable Level - Level 2)

Цьому рівню відповідають підприємства, що володіють певними технологіями управління та розробки. Управління вимогами та планування в більшості випадків ґрунтуються на розробленій документованій політиці та накопиченому досвіді. Встановлено та введено в повсякденну практику базові показники для оцінки параметрів проекту. Менеджери відстежують виконання робіт і контролюють тимчасові і виробничі витрати.

У компанії розроблені деякі внутрішні стандарти і організовані спеціальні групи перевірки якості QA . Зміни версій кінцевого програмного продукту і створених проміжних програмних засобів відслідковуються в системі управління конфігурацією.

Є необхідна дисципліна дотримання встановлених правил. Ефективні методики та процеси впроваджуються, що забезпечує можливість повторення успіху попередніх проектів в тій же прикладній галузі. Ключові галузі процесу розробки ПЗ цього рівня наведені на рисунку

<i>Повторюваний рівень (Repeatable Level)</i>
• Керування вимогами (Requirements management) • Планування проекту розробки ПЗ (Software project planning) • Відстеження ходу проекту та контроль (Software project tracking and oversight) • Керування субпідрядниками розробки ПЗ (Software subcontract management) • Забезпечення якості розробки ПЗ (Software quality assurance) • Керування конфігурацією продукту (Software configuration management).

Визначний рівень (Defined Level - Level 3) Рівень характеризується деталізованим методологічним підходом до управління (описані і закріплені в документованій політиці типові дії, що необхідні для багаторазового повторення: ролі і відповідальність учасників, стандартні процедури та операції, порядок дій, кількісні показники і метрики процесів, формати документів та ін.). У всіх проектах використовується стандартний для організації процес розробки та підтримки програмного забезпечення, адаптований під конкретний проект. Для створення і підтримки методологій в актуальному стані в організації підготовлена і постійно функціонує спеціальна група.

Компанія регулярно проводить тренінги для підвищення професійного рівня своїх співробітників. Починаючи з цього рівня, організація практично перестає залежати від особистісних якостей конкретних розробників і не має тенденції опускатися на більш низькі рівні. Ця незалежність обумовлена продуманим механізмом постановки завдань, планування заходів, виконання операцій і контролю виконання.

Управлінські та інженерні процеси документовані, стандартизовані і інтегровані в уніфіковану для всієї організації технологію створення ПЗ. Кожен проект використовує затверджену версію цієї технології, адаптовану до особливостей поточного проекту.

Ключові галузі процесу розробки ПЗ цього рівня наведені на рисунку

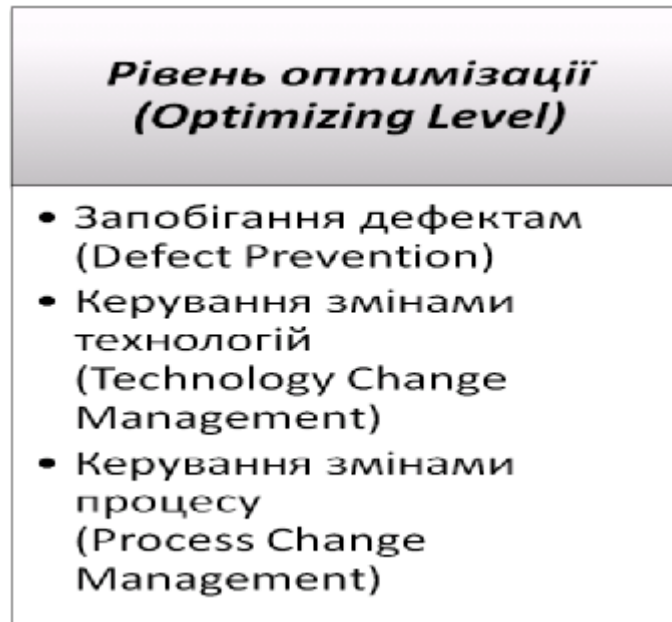
<i>Визначений рівень (Defined Level)</i>
• Мета організаційних процесів (Organization Process Focus) • Визначення (стандартного) процесу (Organization Process Definition) • Програма навчання (Training Program) • Керування інтегрованою розробкою ПЗ (Integrated Software Management) • Технологія розробки програмних продуктів (Software Product Engineering) • Міжгрупова координація (Intergroup Coordination). • Експертні (спільні) оцінки колег (Peer Reviews)

Керований рівень (Managed Level – Level 4)

Рівень, на якому розроблені і закріплені у відповідних нормативних документах кількісні показники якостей. Збираються детальні кількісні дані щодо функціонування процесів розробки та якості кінцевого продукту. Аналізується значення і динаміка цих даних. Більш високий рівень управління проектами досягається за рахунок зменшення відхилень різних показників проекту від запланованих. При цьому тенденції зміни продуктивності процесу можна відокремити від випадкових варіацій на підставі статистичної обробки результатів вимірювань в процесах. Ключові області процесу розробки ПЗ цього рівня наведені на наступному рисунку.



Для цього рівня заходи щодо вдосконалення розраховані не тільки на існуючі процеси, а й на впровадження, використання нових технологій і оцінку їх ефективності. Основним завданням всієї організації на цьому рівні є постійне вдосконалення існуючих процесів, в ідеалі спрямоване на запобігання відомої помилки або дефекту і попередження можливих. Застосовується механізм повторного використання компонентів від проекту до проекту (шаблони звітів, формати вимог, процедури і стандартні операції, бібліотеки модулів програмних засобів). Ключові галузі процесу розробки ПЗ цього рівня наведені на рисунку .



В результаті аудиту та атестації компанії присвоюється певний рівень, за результатами наступних аудитів рівень може підвищуватися або знижуватися. Кожен наступний рівень в обов'язковому порядку включає в себе всі ключові характеристики попередніх. У зв'язку з цим сертифікація компанії в одному з рівнів передбачає безумовне виконання всіх вимог більш низьких рівнів. Документація з повним описом SW-CMM займає близько 500 сторінок і визначає набір з 312 вимог, яким повинна відповідати організація, якщо вона планує атестуватися за цим стандартом на 5-ий рівень зрілості.

До переваг моделі СММ відноситься те, що вона орієнтована на організації, які займаються розробкою програмного забезпечення. У даній моделі вдалося більш детально визначити вимоги, які специфічні для процесів, що пов'язані з розробкою ПЗ. З цієї причини в СММ наведені не тільки вимоги до процесів організації, а й приклади реалізації таких вимог.

Основний недолік СММ полягає в тому, що модель не авторизована як стандарт ні міжнародними, ні національними органами стандартизації. Втім, СММ давно стала промисловим стандартом. До недоліків моделі також необхідно віднести великі зовнішні накладні витрати на приведення процесів компанії у відповідність до моделі СММ, ніж при використанні моделей Міжнародного стандарту ISO 9000.

Стандарт SPICE (Software Process Improvement and Capability Determination)

Найбільше SPICE нагадує CMM. Точно так само, як і в CMM, основним завданням організації є постійне поліпшення процесу розробки можливостей (в SPICE визначено 6 різних рівнів), але ці рівні застосовуються не тільки до організації в цілому, а й до окремо взятих процесів. В основі стандарту лежить оцінка процесів. Ця оцінка виконується шляхом порівняння процесу розробки програмного забезпечення, що існує в даній організації, з описаною в стандарті моделлю. Аналіз результатів, що отримані на цьому етапі, допомагає визначити сильні і слабкі сторони процесу, а також внутрішні ризики, що властиві даному процесу. Це допомагає оцінити ефективність процесів, визначити причини погіршення якості та пов'язані з цим витрати в часі або вартості. Потім виконується визначення можливостей процесу, тобто можливостей його поліпшення. В результаті в організації може з'явитися розуміння необхідності поліпшення того чи іншого процесу. До цього моменту цілі вдосконалення процесу вже чітко сформульовані і залишається тільки технічна реалізація поставлених завдань. Після цього весь цикл робіт починається спочатку.

Наступна модель (методологія) розроблення ПЗ:

Rational Unified Process (RUP) – розробка фірми Rational Software. Rational Unified Process (RUP), що пропонує ітеративну модель розроблення, що включає чотири фази: початок, дослідження, побудову та впровадження. Проходження через чотири основні фази називається циклом розроблення, кожен цикл завершується генерацією працездатної версії системи. У ході супроводу продукт продовжує розвиватися і знову проходить ті самі фази.

Модель RUP описує загальний абстрактний процес, на основі якого організація або проектна команда повинна створити конкретний спеціалізований процес, орієнтований на її потреби.

Уніфікований процес розроблення ПЗ Rational Unified Process (RUP) виник як відповідь на потребу розробників у процесі, що об'єднує безліч аспектів розроблення програм і відповідає таким вимогам:

- забезпечує керівництво діяльністю команди;
- керує завданнями окремого розробника і команди в цілому;
- показує, які артефакти необхідно розробити;
 - надає критерії відстеження та вимірювання продуктів і функціонування проекту.

Уніфікований процес розроблення ПЗ передбачає розподіл ЖЦ програмного забезпечення на фази



- **Фаза початку проекту (Inception).** Визначаються основні цілі проекту, бюджет проекту, основні засоби його виконання - технології, інструменти, ключовий персонал, складаються попередні плани проекту. Основна мета цієї фази - досягти компромісу між усіма зацікавленими особами щодо завдань проекту.
- **Фаза опрацювання (Elaboration).** Основна мета цієї фази - на базі основних, найбільш істотних вимог, розробити стабільну базову архітектуру продукту, яка дозволяє вирішувати поставлені перед системою завдання і надалі використовується як основа розробки системи.
- **Фаза побудови (Construction).** Основна мета цієї фази - детальне прояснення вимог і розробка системи, що задовольняє їм, на основі спроектованої раніше архітектури.
- **Фаза передачі (Transition).** Мета фази - зробити систему повністю доступною кінцевим користувачам. Тут відбувається остаточне розгортання системи в її робочому середовищі, підгонка дрібних деталей під потреби користувачів. В рамках кожної фази можливе проведення декількох ітерацій, виконуваного проекту.

Діяльності (основні процеси) RUP діляться на п'ять робочих і чотири підтримуючі. До робочих діяльностей відносяться:

- **Моделювання предметної області** (бізнес-моделювання, Business Modeling). Цілі цієї діяльності - зрозуміти бізнес-контекст, в якому повинна буде працювати система (і переконатися, що всі зацікавлені особи розуміють його однаково), зрозуміти можливі проблеми, оцінити можливі їх рішення та їх наслідки для бізнесу організації, в якій працюватиме система.
- **Визначення вимог** (Requirements). Цілі - зрозуміти, що повинна робити система, визначити межі системи і основу для планування проекту та оцінок ресурсозатрат в ньому.
- **Аналіз та проектування** (Analysis and Design). Вироблення архітектури системи на основі ключових вимог, створення проектної моделі, представленої у вигляді діаграм UML, що описують продукт з різних точок зору.
- **Реалізація** (Implementation). Розробка вихідного коду, компонент системи, тестування та інтегрування компонент.
- **Тестування** (Test). Загальна оцінка дефектів продукту, його якість в цілому; оцінка ступеня відповідності вихідним вимогам.

Підтримуючими діяльностями є:

- **Розгортання (Deployment).** Цілі – розгорнути систему в її робочому оточенні і оцінити її працездатність.
- **Управління конфігураціями і змінами (Configuration and Change Management).** Визначення елементів, що підлягають зберіганню та правил побудови з них узгоджених конфігурацій, підтримку цілісності поточного стану системи, перевірка узгодженості внесених змін.
- **Управління проектом (Project Management).** Включає планування, управління персоналом, забезпечення зв'язків з іншими зацікавленими особами, управління ризиками, відстеження поточного стану проекту.
- **Управління середовищем проекту (Environment).** Налаштування процесу під конкретний проект, вибір і зміна технологій та інструментів, що використовуються в проекті.

Методологія Microsoft Solution Framework (MSF)

розробка фірми Microsoft, призначена для вирішення широкого кола завдань. Технологія масштабована, тобто налагоджується на вирішення завдань будь-якої складності колективом будь-якої чисельності. Microsoft Solution Framework (MSF) – методологія, що розроблялася для підвищення керованості процесів розроблення окремого підприємства (Microsoft), а потім стала застосовуватися розробниками, які використовують продукти Microsoft. Методологія, подібна до RUP, також включає чотири фази: аналіз, проектування, розроблення, стабілізацію, є ітераційною, припускає використання об'єктно-орієнтованого моделювання. MSF порівняно з RUP здебільшого орієнтована на розроблення бізнес-додатків.

В основу MSF покладено вісім базових принципів (foundational principles):

- **Сприяння відкритій комунікації** (Foster open communications). Модель процесу MSF запроваджує вільний інформаційний потік серед членів команди та зацікавлених сторін (key stakeholders) для забезпечення однакового розуміння завдань. Документування ходу проекту та доступ до цих даних членів команди, зацікавлених сторін та замовників.
- **Робота у напрямку спільного бачення проекту** (Work toward a shared vision). Модель процесу MSF запроваджує фазу формування концепції (Envisioning Phase) та окремий ключовий момент затвердження бачення (milestone Vision/Scope Approved) для формування спільного бачення проекту. Бачення включає детальне розуміння цілей та завдань для досягнення рішення поставленої проблеми. Спільне бачення виявляє припущення команди та замовників, потрібні для отримання рішення.

- **Надання прав і можливостей членам команди** (Empower team members). Розширення прав і можливостей членів команди для прийняття ними відповідальності за виконану роботу. Таке збільшення відповідальності може бути затверджене у графіках, де фіксується дата закінчення робіт, що також може бути засобом виявлення можливих затримок проекту.
- **Визначення індивідуальної та спільної відповідальності** (Establish clear accountability and shared responsibility). Модель командної групи MSF ґрунтується на принципі важливості роботи кожного для отримання якісного рішення проблеми. Усі члени групи розділяють відповідальність за проект.

- Зосередження на бізнес-цілях (Focus on delivering business value).

Рішення повинне приносити користь організації у вигляді додавання вартості бізнесу. Це додавання досягається тільки після повного розгортання рішення у виробничому середовищі.

– Бути гнучкими, очікувати на зміни (Stay agile, expect change).

MSF припускає, що у виробничому середовищі на рішення постійно впливають зміни. Команда повинна бути обізнаною та готовою до керування змінами вимог.

- **Інвестування у якість** (Invest in quality). У MSF кожен член команди відповідальний за якість вирішення завдання. Для підтримки якості протягом проекту формується команда тестувальників. Це гарантує, що рішення відповідає рівню якості, визначеному зацікавленими сторонами.
- **Навчання за досвідом** (Learn from all experiences). MSF вимагає використання досвіду, отриманого у попередніх проектах. Це дозволяє знайти найкращі методики розроблення.

MSF складається із двох моделей та трьох дисциплін:

- моделі командної групи;
- моделі процесу;
- дисципліни управління проектами;
- дисципліни управління ризиками;
- дисципліни управління підготовкою.

Модель командної групи (MSF Team Model) описує, як організувати колективи і якими принципами керуватися для досягнення максимального успіху в розробленні програм. Різні колективи можуть по-своєму застосовувати на практиці різні елементи цієї моделі – все залежить від масштабу проекту, розміру колективу і кваліфікації його учасників та моделі процесу розроблення.

Модель командної групи визначає тільки ролі, кожна з яких може виконуватися кількома людьми. У моделі командної групи не передбачено єдиноначальності – усі ролі важливі, усі ролі рівноправні, тому MSF називають моделлю рівних (team of peers).



- **Program management** – керування програмою. Виконавець цієї ролі відповідає за організацію (але не керує): здійснює ведення графіка робіт, ранкові 15-хвилинні наради, забезпечує відповідність стандартам і специфікаціям, фіксацію порушень, написання технічної документації.
- **Product management** – керування продуктом. Виконавці цієї ролі відповідають за спілкування із замовником, написання специфікації, роз'яснення завдань розроблювачам.
- **Development** – найбільш традиційна роль – розроблення і початкове тестування продукту.
- **User experience** – підвищення ефективності роботи користувачів, написання користувальницької документації.
- **Release management** – розгортання релізу продукту, супровід і його технічна підтримка.
- **Test** – визначення відповідності показників якості релізу встановленим значенням. Виявлення й усунення недоробок, виправлення помилок, інші функції QA.

Модель процесу (MSF Process Model) визначає, коли і які роботи повинні бути виконані. Основні принципи і практичні прийоми, на яких ґрунтується модель:

- ітеративний підхід (послідовний випуск версій);
- підготовка чіткої документації;
- урахування невизначеності майбутнього;
- облік компромісів; — керування ризиками;
- підтримка відповідального відношення колективу до строків випуску продукту;
- розбивка великих проектів на більш дрібні керовані частини;
- щоденне складання проекту;
- постійний аналіз ходу робіт.

Process model має три основні особливості:

- розбивка всього процесу на фази;
- використання опорних точок (milestones);
- ітеративність.

Увесь процес розбивається на п'ять взаємозалежних фаз. Перш ніж переходити до наступної фази, на попередній повинні бути отримані певні результати (досягнуті головні опорні точки). Фактично процес ітеративний і відповідає спіральній моделі ЖЦ ПЗ. У моделі передбачається наявність основних віх (завершення головних фаз моделі) і проміжних, що відображають внутрішні етапи головних фаз.



Створення загальної картини додатку (Envisioning).

На цьому етапі вирішуються такі основні завдання: – оцінка існуючої ситуації; – визначення складу команди; – структури проекту, бізнес-цілей, вимог і профілів користувачів; – розробка концепції рішення і оцінка ризику.

Планування (Planning). Включає планування і проектування продукту. На основі аналізу вимог розробляється проект і основні архітектурні рішення, функціональні специфікації системи, плани і календарні графіки, середовища розробки, тестування та пілотної експлуатації.

Розробка (Developing). Створюється варіант вирішення проблеми, у вигляді коду і документації чергового прототипу, включаючи специфікації і сценарії тестування. Основна віха етапу – "Остаточне затвердження області дії проекту". Продукт готовий до зовнішнього тестування і стабілізації. Крім того, замовники, користувачі, співробітники служби підтримки та супроводу, а також ключові учасники проекту можуть попередньо оцінити продукт і вказати всі недоліки, які потрібно усунути до його поставки.

- **Стабілізація** (Stabilizing). Підготовка до випуску остаточної версії продукту, доведення його до заданого рівня якості. Тут виконується комплекс робіт з тестування (виявлення та усунення дефектів), перевіряється сценарій розгортання продукту. Коли рішення стає досить стійким, проводиться його пілотна експлуатація в тестовому середовищі із залученням користувачів і застосуванням реальних сценаріїв роботи.
- **Розгортання** (Deploying). Виконується установка рішення і необхідних компонентів оточення, проводиться його стабілізація в промислових умовах і передача проекту групі супроводу. Крім того, аналізується проект в цілому на предмет рівня задоволеності замовника.

- Важливу роль відіграють **опорні точки (milestones)**, у яких аналізується стан робіт і виробляється їхня синхронізація. У цих точках додаток або його специфікації не заморожуються. Опорні точки дозволяють проаналізувати стан проекту і внести необхідні корективи, наприклад, переналагоджуватися під вимоги, що змінилися, замовника або відреагувати на ризики, можливі в ході подальшої роботи. Для кожної опорної точки визначається, які результати повинні бути отримані до цього моменту.
- Кожна фаза процесу розроблення завершується головною опорною точкою (major milestone) – моментом, коли всі члени колективу синхронізують отримані результати. Призначення таких точок у тому, що вони дозволяють оцінити життєздатність проекту. Їхні результати видимі не тільки колективу розроблювачів, але й замовникові. Після аналізу результатів колектив розробників і замовник спільно вирішують, чи можна переходити на наступну фазу. Таким чином, головні опорні точки - це критерії переходу з однієї фази проекту на іншу.

Методологія eXtreme Programming (XP)

Екстремальне програмування (eXtreme Programming, XP) – спрощена методологія організації виробництва для невеликих і середніх за розміром команд розробників, які займаються розробленням програмного продукту в умовах незрозумілих або швидко змінних вимог. Програмування відповідно до методик XP доводить використання загальноприйнятих принципів програмування до екстремальних рівнів:

- перегляд коду виконується постійно (з урахуванням того, що програмування ведеться парами);
- кожен учасник проекту тестує код програми постійно (тестування модулів), навіть замовники проводять функціональне тестування;
- проектування є складовою частиною повсякденної роботи кожного розробника (перероблення коду);
- розроблення виконується з урахуванням вимоги збереження в системі найбільш простого дизайну, що забезпечує поточний необхідний рівень функціональності (простіші речі надійніші в роботі);
- увага до архітектури системи на кожному етапі проекту;
- інтеграційне тестування виконується після кожної найменшої зміни у системі (триваюча інтеграція);
- ітерації невеликі – тривають години, кілька днів (постійне планування).

Для формування стилю розроблення, що дозволить досягти потрібної якості рішення, в XP пропонується керуватись чотирма цінностями:

- **комунікацією** (communication) – дисципліна XP, спрямована на забезпечення безперервної комунікації усіх учасників проекту. Під час тестування, програмування в парах та попереднього оцінювання робіт замовники, розробники та менеджери змушені постійно спілкуватись;
- **простотою** (simplicity) – неможливо заздалегідь точно визначити, як буде розвиватися проект, тому вирішувати необхідно лише завдання, що виникають сьогодні, у найбільш простий спосіб;

- **зворотним зв'язком** (feedback) – спосіб отримати точні та конкретні дані про стан проекту. Постійне виконання тестів для перевірки усіх змін у системі забезпечує програміста звороним зв'язком про якість роботи. Кожен новий опис замовником вимог до системи одразу ж оцінюється розробниками і забезпечує замовника зворотним зв'язком із інформацією про якість опису. Менеджер, який контролює строки виконання робіт, забезпечує усіх учасників проекту інформацією про виконання планових термінів розроблення. В ХР найбільш важливі функції системи реалізуються в першу чергу у реально працюючий продукт, тому замовник досить швидко може оцінити хід виконання та якість розроблення, а також відповідність замовленню;
- **хоробрість** (courage) – для того, щоб розроблення не втратила своєї актуальності, в ХР рішення повинні прийматися з максимальною швидкістю, для чого потрібна певна сміливість.

Для визначення методів вирішення проблем розроблення ПЗ на основі обраних цінностей потрібно керуватися такими фундаментальними принципами оцінки методів вирішення поставленого завдання:

- швидкий зворотний зв'язок – інформація про стан системи повинна якомога швидше надходити до зацікавлених сторін проекту. У рамках дисципліни XP ці дані повинні із максимальною швидкістю надходити, інтерпретуватися та на основі їх аналізу швидко виконуватися модифікації системи;
- прийнятна простота – кожна проблема повинна вирішуватись у найбільш простий спосіб. У рамках XP значні зусилля (тестування, переробка коду, комунікації) покладаються для вирішення завдань сьогодення таким чином, щоб завтра внесення змін не потребувало великих зусиль;
- поступова зміна – кардинальні зміни часто приводять до провалу проекту, щоб заплановані модифікації мали успіх, їх потрібно реалізовувати як серію невеликих змін, після кожної з яких перевіряється працездатність системи;
- прийнятна зміна – найвигідніша стратегія – та, що дозволяє вирішити найбільш важливу проблему і залишає максимальну свободу дій;
- якісна робота – кожен учасник проекту повинен прагнути максимально якісно виконувати свої завдання і сприяти якісній роботі інших.

Крім того, в ХР є кілька правил (технік), що характеризують особливості моделі його життєвого циклу:

- **Живе планування** (planning game) – якнайшвидше визначити обсяг робіт, який потрібно зробити до наступної версії ПЗ. Рішення приймається на основі, в першу чергу, бізнес-пріоритетів замовника і, по-друге, технічних оцінок. Плани змінюються, як тільки вони починають розходитися з дійсністю або побажаннями замовника.
- **Часта зміна версій** (small releases) – перша працююча версія повинна з'явитися якомога швидше, і тут же повинна почати використовуватися. Наступні версії готуються через досить короткі проміжки часу.

- **Прості проектні рішення** (simple design) – у кожен момент часу система повинна бути сконструйована так просто, наскільки це можливо. Нові функції додаються тільки після ясного прохання про це. Вся зайва складність вилючається, як тільки виявляється.
- **Розробка на основі тестування** (test-driven development) - спочатку пишуться тести, потім реалізуються модулі так, щоб тести спрацьовували. Замовники заздалегідь пишуть тести, що демонструють основні можливості системи, щоб можна було побачити, що система дійсно запрацювала.
- **Постійна переробка** (refactoring) - системи для усунення зайвої складності, збільшення зрозумілості коду, підвищення його гнучкості. При цьому перевага віддається більш елегантним і гнучким рішенням, у порівнянні з тими, які просто дають потрібний результат.

- **Програмування парами (pair programming)** - весь код пишеться двома програмістами на одному комп'ютері, що підвищує його якість (відсутність помилок, зрозумілість, читання, ...).
- **Постійна інтеграція (continuous integration)** - система збирається і проходить інтеграційне тестування якомога частіше, по кілька разів на день кожного разу, коли пара програмістів закінчує реалізацію чергової функції.
- **40-годинний робочий тиждень** - понаднормова робота розглядається як ознака великих проблем у проекті. Не допускається понаднормова робота 2 тижні поспіль - це виснажує програмістів і робить їх роботу значно менш продуктивною.

- **PSP/TSP**

- Одна з останніх розробок Інституту програмної інженерії Personal Software Process / Team Software Process [12,13]. Personal Software Process визначає вимоги до компетенцій розробника. Згідно цієї моделі кожен програміст повинен вміти:

- враховувати час, витрачений на роботу над проектом;
- враховувати знайдені дефекти;
- класифікувати типи дефектів;
- оцінювати розмір задачі;
- здійснювати систематичний підхід до опису результатів тестування;
- планувати програмні завдання;
- розподіляти їх по часу і скласти графік роботи.
- виконувати індивідуальну перевірку проекту та архітектури;
- здійснювати індивідуальну перевірку коду;
- виконувати регресійне тестування. Team Software Process робить ставку на самокеровані команди чисельністю 3-20 розробників.

Команди повинні: • встановити власні цілі; • скласти свій процес і плани; • відслідковувати роботу; • підтримувати мотивацію і максимальну продуктивність. Послідовне застосування моделі PSP/TSP дозволяє зробити нормою в організації п'ятий рівень CMM.

Гнучке розроблення ПЗ на основі Agile

Основна ідея всіх гнучких моделей полягає в тому, що застосовуваний у розробці процес повинен бути адаптивним. Вони декларують своєю вищою цінністю орієнтованість на людей і їх взаємодію, а не на процеси і засоби. По суті, так звані, гнучкі методології це не методології, а набір практик, які можуть дозволити (а можуть і ні) домагатися ефективної розробки ПЗ.

Основні принципи гнучкого розроблення ПЗ

Найвищий пріоритет – задоволення потреб замовника шляхом завчасного та регулярного постачання програмного забезпечення.

Схвальне ставлення до змін, навіть на завершальних стадіях розроблення. Agile-процеси використовують зміни для забезпечення конкурентоспроможності замовника.

Працюючий продукт необхідно випускати якомога частіше, з періодичністю від двох тижнів до двох-трьох місяців.

Упродовж усього проекту розробники і представники бізнесу повинні працювати разом щодня.

Над проектом повинні працювати вмотивовані професіонали, які працюють у зручних умовах, із повною підтримкою та довірою менеджменту проекту.

Особиста комунікація – найефективніший та найпрактичніший метод обміну інформацією в команді.

Працюючий продукт – головний показник прогресу.

Інвестори, розробники і користувачі повинні мати можливість підтримувати постійний ритм роботи. Agile допомагає налагодити такий сталий процес розроблення.

Постійна увага до технічної досконалості і якості проектування підвищує гнучкість проекту.

Простота – мистецтво мінімізації зайвої роботи – дуже необхідна.

Найкращі вимоги, архітектурні та технічні рішення виникають у командах, які здатні самоорганізовуватись.

Команда постійно шукає способи підвищення ефективності та відповідно коригує свою роботу.

Потрібно виділити фактори, які можуть **негативно** вплинути на використання гнучких методів:

- масштабні зусилля у галузі розвитку (більше 20 розробників);
- розподілена у просторі команда;
- примусове впровадження гнучкого процесу усупереч вимогам команди розробників;

Небажане використання agile-методів у критично важливих системах, де відмова ПЗ неможливий ні в якому разі (наприклад, управління повітряним рухом). Agile акцентує увагу на безпосередньому спілкуванні між учасниками проекту. Більшість agile-команд розміщені в одному офісі (bullpen). До команди обов'язково включають представників замовника (замовники, які визначають продукт, менеджери продукту, бізнес-аналітики або користувачі). Також до команди входять тестувальники, дизайнери інтерфейсу, технічні автори та менеджери.

Важкі і легкі моделі виробничого процесу мають свої переваги і свої недоліки

Таблиця 1. Плюси і мінуси важких і легких моделей процесів розробки ПЗ

Вага моделі	Плюси	Мінуси
Важкі	Процеси розраховані на середню кваліфікацію виконавців. Більша спеціалізація виконавців. Нижче вимоги до стабільності команди. Відсутні обмеження за обсягом і складністю виконуваних проектів.	Вимагають істотної управлінської надбудови. Більш тривалі стадії аналізу і проектування. Більш формалізовані комунікації.
Легкі	Менше непродуктивних витрат, пов'язаних з управлінням проектом, ризиками, змінами, конфігураціями. Спрощені стадії аналізу і проектування, основний упор на розробку функціональності, поєднання ролей. Неформальні комунікації.	Ефективність сильно залежить від індивідуальних здібностей, вимагають більш кваліфікованої, універсальної і стабільної команди. Обсяг і складність виконуваних проектів обмежені.

Це означає, що не існує єдиного правильного процесу розробки ПЗ, в кожному новому проекті процес повинен визначатися щоразу заново, в залежності від проекту, продукту та персоналу

ДЯКУЮ

